From NobleCount to the GPU

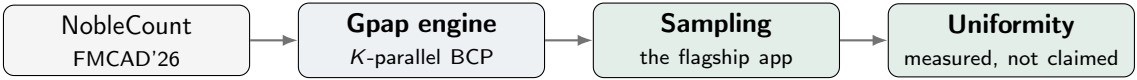
A K-parallel BCP primitive, and guaranteed-valid SAT solution sampling

Yogev Shalmon

Technion — Faculty of Data and Decision Sciences

Project status — August 2, 2026

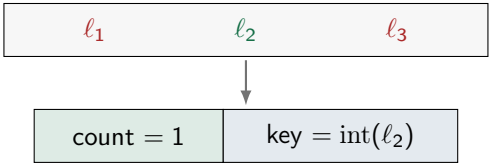
The story



The data structure is the thread: what removed clause scans from a CPU flip is what makes the GPU update race-free.

Where this starts: NobleCount (FMCAD'26)

a clause, one literal currently true



two fields per clause, updated branch-free on every flip

count $= 1 \Rightarrow$ the key **is** the surviving literal
$\Rightarrow$ **no clause scan, ever**

Dropped into ProbCCAnr, the SOTA local-search solver. **No heuristic changes, no re-tuning.**

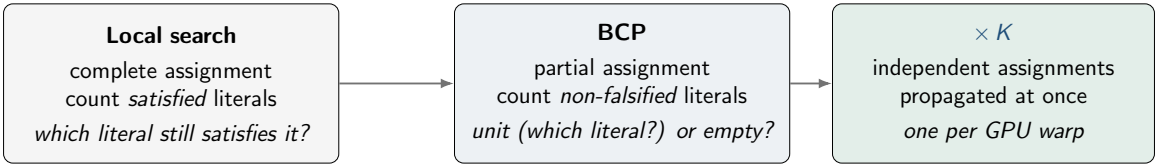
New state of the art on **FCC** (PAR2 -24.8%) *and* **SC22**.

A data-structure win, orthogonal to heuristic design.

Its own closing sentence

"... making it **a promising candidate for parallel clause-state maintenance in SIMD- or GPU-based implementations**."

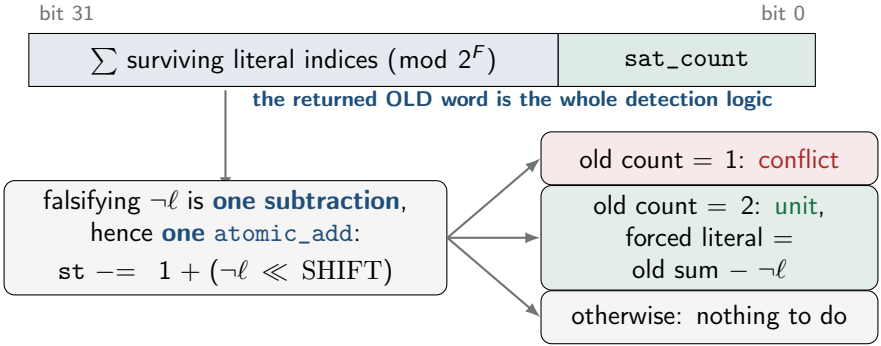
Same idea, different question



The primitive, in one line

Negated occurrence lists: setting ℓ true falsifies $\neg\ell$ in *exactly* the clauses of $\mathrm{occ}[\ell]$ — a decision touches only what it can affect.

One word per clause — and the update is its own detector



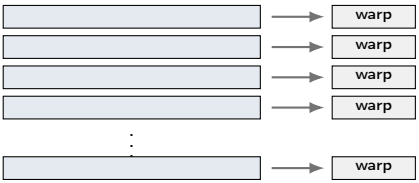
The **assignment** is the same trick: a bitmap where ℓ and $\neg\ell$ share a word, so one `atomic_or` returns duplicate *and* contradiction together.

On a CPU, XOR and sum are equivalent — NobleCount used XOR. On a GPU they are not: the sum makes the update an atomic whose *return value* resolves every race — no locks, no `shfl`/`ballot`.

The architecture: one assignment per warp

state: K independent rows

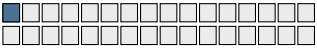
clause words (M) + assignment bits ($2N$)



a row is program-private

$\Rightarrow$ **no cross-row races**

SCALAR — small formulas, huge K



1 lane walks $\mathrm{occ}[\ell]$, 31 idle — no atomics needed: thousands of rows already saturate DRAM.

WAVE — large formulas, K-starved

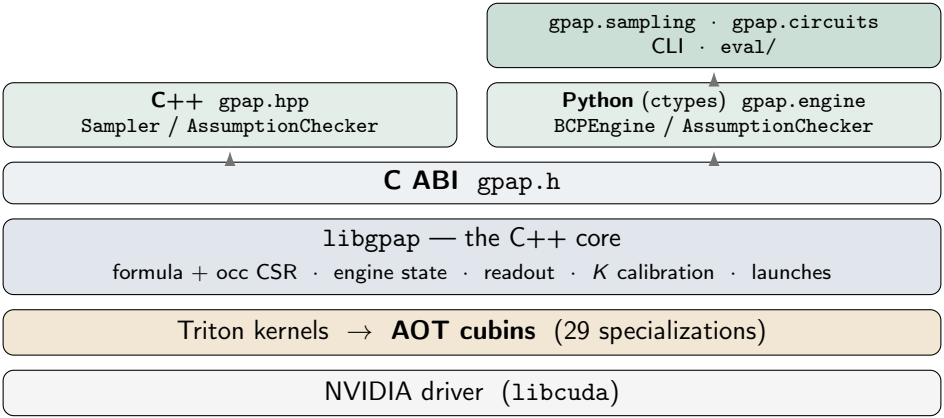


32 clause updates in flight *inside one assignment*; the atomics resolve the lane races.

Auto-dispatch: WAVE while $K < 30\times$ the GPU's resident warps, scalar beyond it

VRAM caps K at hundreds of rows on big formulas, so each row is latency-starved — that is what the extra lanes are for. Both variants share the state and give the same results.

The tool



One implementation: both faces link the same library, so there is no second engine to drift. The build needs no CUDA toolkit and no `nvcc`; the runtime needs `libcuda` alone.

The engine, in features

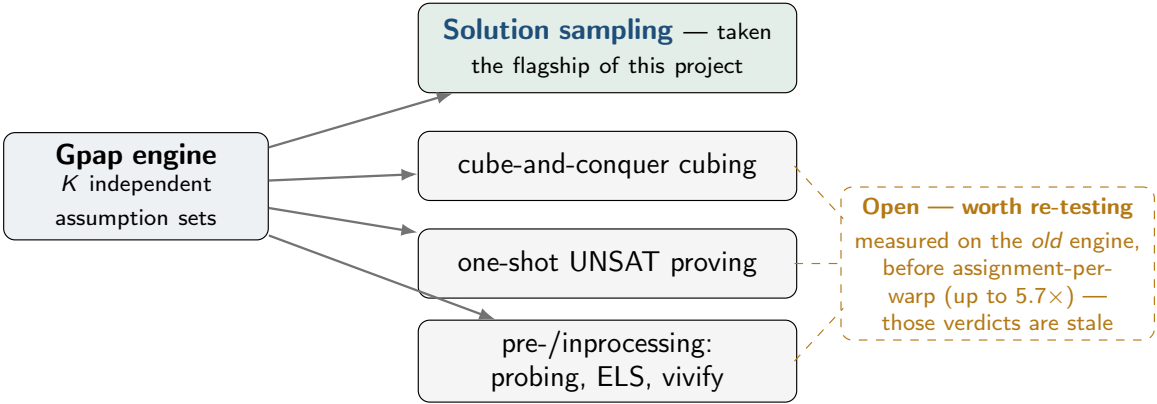
- one packed `int32` per clause — update is a single atomic
- assignment as a bitmap — one atomic gives dedup *and* contradiction
- two propagate kernels, **auto-dispatched** per formula
- the whole decide+propagate trajectory **fused into one launch**
- K sized analytically from free VRAM; unit-flush cached at construction
- readout stays **on device** — only counts and projections cross the bus
- two packed splits chosen per formula; out-of-envelope formulas rejected, never silently degraded

Bottom line

Up to 5.7× end-to-end over the pre-WAVE engine — with **bit-identical model sets** at every step.

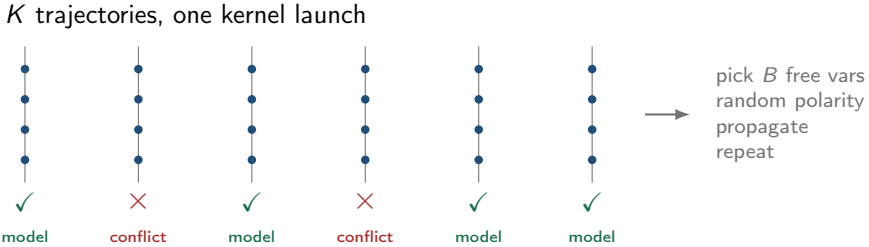
Negative results, kept because they cost weeks: classic **2-watched-literals on GPU is 1.8–2.6× slower** than NobleCount-style counting, on ∼5× the per-row state; 1-byte clause state, clause renumbering and multi-warp tiles all measured out.

What is a K-parallel BCP primitive good for?



Sampling fits the layout exactly: millions of independent randomized completions over one shared, read-only occurrence structure.

Gpap = GPU-parallel CMSGen



CMSGen's Algorithm 1, exactly — but K at a time on the GPU.

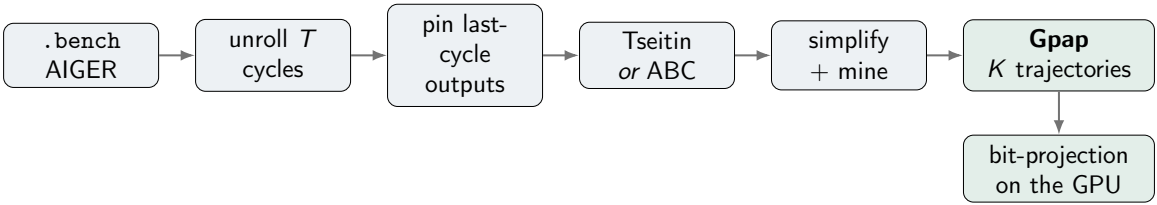
One structural difference: CMSGen backtracks; Gpap *discards*.

Two consequences

Every emitted row is a real model — no gradient, no rounding, no repair.

Discarding *is* rejection sampling $\Rightarrow$ an exactness knob (later).

The flagship pipeline

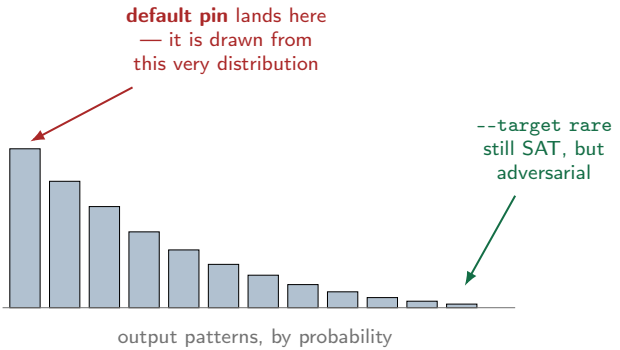


Support = primary inputs × cycles. Only the bit matrix leaves the device.

Encoder is a choice: native Tseitin, or ABC. Same projected solution set — gated by set-equality tests.

Config is per instance: a bandit measures at full K. The right answer is instance-dependent, so we never guess it.

The question we asked before a reviewer could: why not just simulate?



At default pins, CPU rejection beat us on 17/19 — so we changed the protocol, not the claim.

We use rejection, we do not just compete with it: it picks the targets, it audits every emitted sample, and at $B \geq |\text{support}|$ our sampler *is* rejection sampling.

And there are instances where it simply cannot play — EPFL at rare targets: 0 hits in ∼10M random patterns while Gpap holds 87k–64M samples/s at 89–100% success.

Also fixed before publishing: vacuous output pins (b02/b05/b07/s526 are constant at $T{=}25$), CMSGen timed *with* sample emission (5.3× its wall), and a GPU clock that swings 615–1710 MHz mid-run ⇒ interleaved A/Bs, 3-seed medians.

Headline: 357 rows, 13 stages, 0 errors

family	targets	GPAP / CMSGen	wins
ITC'99 + ISCAS-89 sequential, T=25	**rare**	47.7×	17/18
ITC'99 + ISCAS-89 sequential, T=25	random	107.9×	18/18
ISCAS-85 combinational	**rare**	538×	10/10
HWMCC'17 (curated), T=25	**rare**	156×	7/7
EPFL combinational	**rare**	12.7×	13/17
Sketch / `sk_` projected CNFs	—	48.5×	27/29

RTX 4080 laptop vs. single-core patched CMSGen; per instance at its best encoder. Rare targets cost ∼4× of the ratio versus rejection-easy random pins — **quote the rare column**. Every row carries its engine provenance (packed split, propagate variant, K).

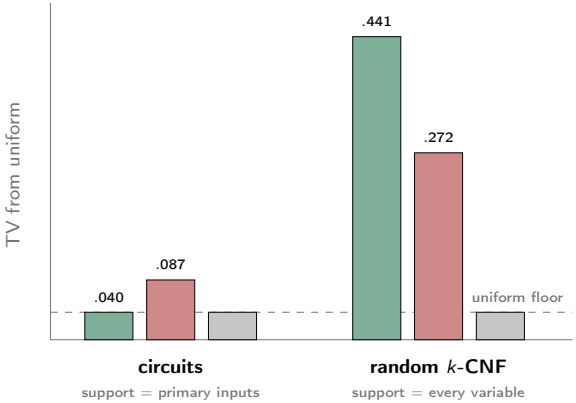
Versus the GPU gradient samplers

rival	on *their* pins	on **rare** pins
HTS (arXiv 2502.08673)	GPAP 26× ahead	0–17 solutions found, total
DEMOTIC (ASP-DAC'25)	their own targets	≈12,000× behind
Recurrent CircuitSAT (MLCAD'25)	wins 3 instances	10^2–10^5× behind

Their pins are rejection-easy — so the ranking flips once targets get rare. All three run on *our* hardware, with *their* metric and *their* protocol; their timings exclude model build, ours include construction.

Honest losses: `75-10`, `prod-20`, the "service" family — jointly-constrained residuals that need clause *learning*, which gradient methods survive and we do not.

Uniformity: measured against an exact oracle



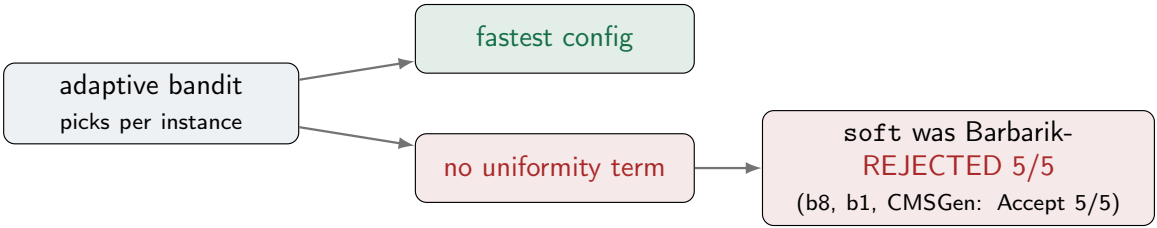
green = Gpap, red = CMSGen, grey = exact-uniform control.

The instrument: generate small formulas, enumerate their solutions exhaustively (solver-free), test χ^2 + TV against the *known* uniform. Calibrated: 0/120 false positives on an exact-uniform control, 97% power at TV 0.05. 600 instances, 10,800 runs, zero unsound rows.

On the flagship regime we are *at* the floor — and CMSGen is not. On random k-CNF we are worse. **Never quote the aggregate.**

An exactness knob: at $B \geq |\text{support}|$ one round fixes the whole support and we discard failures — exact rejection sampling, 24/24 at TV 0.0392. Bounded by yield when p_{accept} is small.

The uncomfortable one: our own bandit optimizes the wrong thing



Acted on: `soft` removed from the default portfolio, opt-in behind `--allow-soft`.

Nothing unsound — every row is a real solution — but a throughput bandit must not pick a rejected distribution for someone who asked for a *sampler*.

The price is large, and this is parked, not settled:

- geomean 2.44× slower on the seq suite
- `b11` 11,269 models → 0; `b12` 50,434 → 789
- same story one level up: the bandit picks $B{=}1$ (faster, more skewed) on 30/30

Status, and what is open

Done

- engine: packed kernels, scalar + WAVE, `libgpap`, three language faces
- sampler: pipeline, portfolios, mining, ELS, GPU projection
- front-ends: `.bench`, AIGER, ABC, simulation
- evaluation: resumable sweep, 3-seed medians, fair CMSGen, 3 GPU rivals
- uniformity: 3-tier readout, Barbarik, exact-oracle fuzzing

Open — build

- **uniformity-aware config choice** — the one item that changes what a user gets
- per-instance encoder auto-pick
- state arena ($\sim$1.2$\times$)
- ROCm port (no kernel work)
- re-test the other applications

Open — measure

- **A100 rerun** — every number is K-bound on a 12 GB card
- multi-core CMSGen, QuickSampler
- break-even simulation throughput
- Barbarik $\times$10 / ScalBarbarik
- one single-session ablation

Turning this into a paper

A GPU-native clause-state primitive	one atomic's return value *is* the unit/conflict detection
A batched-assumption BCP engine	K independent assignments, one per warp + a measured dispatch rule
A sampler	guaranteed-valid; 12–538× over CMSGen, 10^2–10^5× over GPU rivals at rare targets
A measurement standard	rejection baseline, target rarity, vacuous pins, emission fairness, uniformity

Have: every figure on these slides, the 357-row sweep, three rival tables, the uniformity campaign.
Need: the A100 column, a multi-core CMSGen row, one single-session ablation, and a decision on the bandit/uniformity trade.

Backup: numbers on hand

Engine

- vs pre-WAVE: b14 5.7×, b12 2.8×, b04 1.8×, b03 1.5×
- rates: b03 631k, b06 945k, b04 81k, b12 50k, b14 4.0k attempts/s
- C++ port: 1.013× geomean, identical model sets
- portability: 75/1273 CUDA lines, 18 driver calls, 29/29 specializations cross-compile

Encoder

- ABC shrink 2.2–7.8× clauses; at rare targets bimodal (unlocks b14, collapses b13)
- `fast` vs `full`: b15 encode 99 s → $\leq$4 s

Uniformity

- 600 instances, 10,800 runs, 0 unsound; controls 0/600 FP, 600/600 power
- $B \geq |\mathrm{sup}|$: 24/24 ok, TV 0.0392 (theory floor 0.0399)
- circuits 0.040 vs CMSGen 0.087; CNF 0.441 vs 0.272
- `soft`: Reject 5/5 in 156–936 samples, TV median 0.694

Baselines / method

- CMSGen emission = 5.3× its wall (373 MB on 27.sk)
- rare-target amplification 500–500,000×
- SM clock 615–1710 MHz while >90% busy; draw noise ±40%